Date / / Page no.

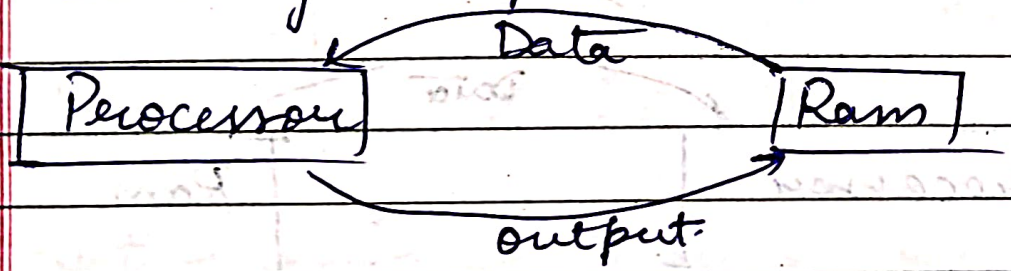
Assembly language

Introduction to Assembly language

programming Language

- Low level
  - Machine lang
    - eg- 1010101001101 = ADD
  - Assembly language
    - we use translator to convert this language into M lang
- High level (converted to M-lang using interpreter or compiler)

Basic of computer Architecture



Busses in Assembly language

- Busses are electrical paths to exchange data from one place to another.

Types

- Internal
  - Data
  - control Read/write.
  - ~~Address~~ (100) ~~Read/write~~
- external

DELTA · My NoteBook

Date / /

Page no.

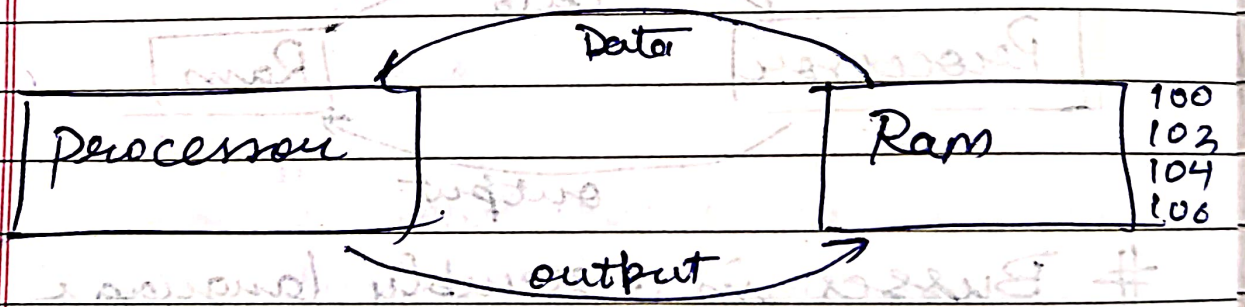
Let us Study ↓ first

Communication modes.

1. Simplex → one way
   one device is.
2. Half Duplex → only able to send at one time
   eg → walkie talkie
3. Full Duplex → Send and Receive data at same time
   eg → mobile phones

→ Note:— • when we need to transfer Data b/w processor and ram there is internal busses used.

• and is b/w Ram and I/O Device, external busses is used.



example:— • ~~If~~ Address bus is used to identify the place of data.

• The ~~the~~ for operation we use control bus.

• For transferring data bus is used.

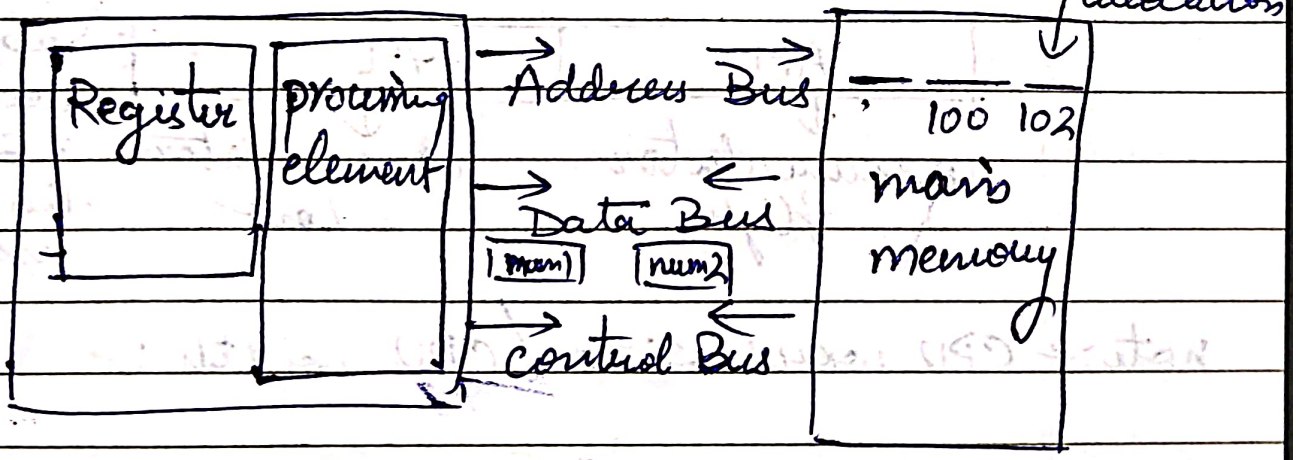
Date / / Page no.

Internal Busses

- Data — half Duplex
- control — Read/write, Status msg, error msg. (half Duplex)
- Address — 100 (Simplex)

Registers in Assembly language.

Why Registers.



Note:- ~~when use~~ • Suppose we have to add two numbers.

- num1 + num2.
- But processor can only ask for a data at a time.
- So processor read num1 and store it in Register, then it read num2 from Ram and num1 from Reg and perform the operation.

ELTA My NoteBook

Date / /

Page no.

Types of Registers

General purpose

Special purpose

General purpose.

1. AX Accumulator
2. BX Base.
3. CX count
4. DX Destination

AX → 16 bit

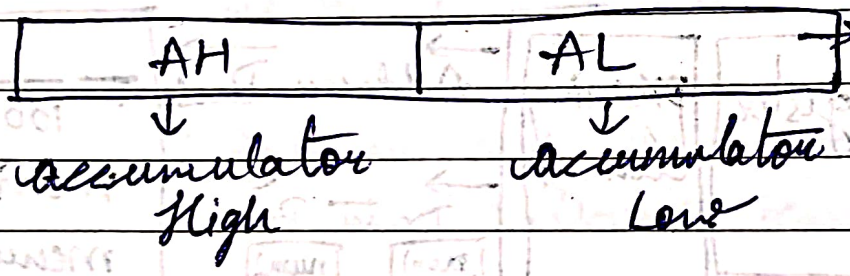
EAX → 32 bit

↓

extended Axa

RAX → 64 bit

X ⇒ Size.



Suppose we need to save a 6 bit num then AL will be used.

note:- CPU word size / CPU width :-

- It means how many no of bits that can be processed by CPU

- Accumulator used in input / output as well as to hold result.
So we can say it is responsible for Airthematic and logical operations.

Date / /

Page no.

2. Base Register :- used to store Base address of data structure.

3. counter Register :- used to track count variable.

eg- for(i=0; i<=10; i++)

↓

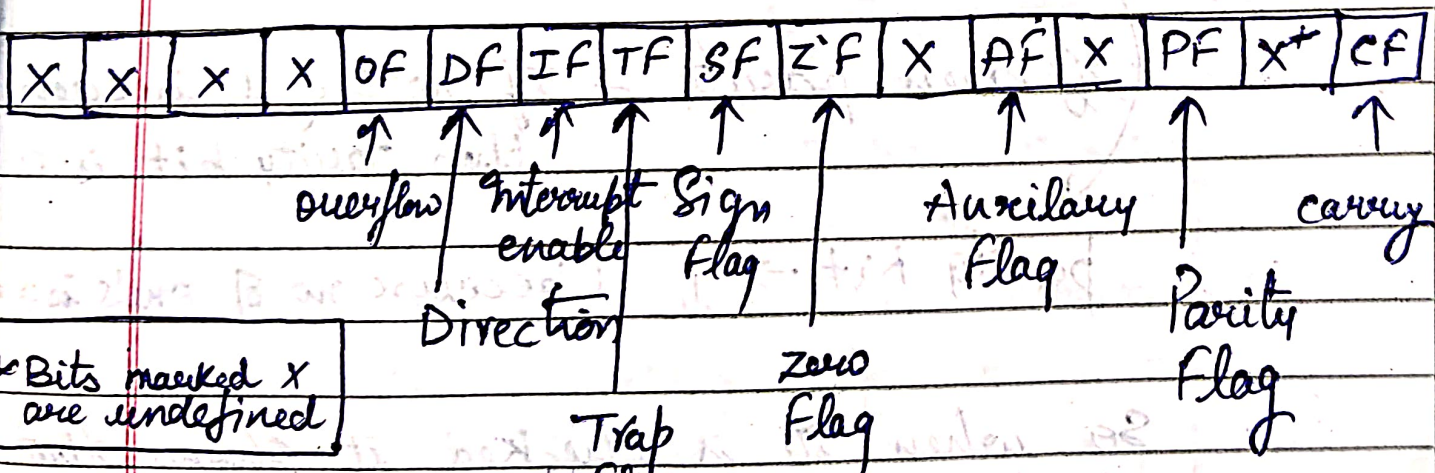
here counter Register take care of this variable i.

4. Destination Register :-

example- num1 * num2 = 38 bits

so if size of Ax is 32 bits then ~~bits~~ a 38 bit number can't be saved into it.

FLAG Register



6 are status Flag

3 are control Flag.

DELTA My NoteBook

Date / /

Page no.

① carry Flag

carry → (1)

```
  1 1 0 0 1 0 0 0
+ 1 1 0 1 0 0 1 0
-----------------
1 1 1 0 1 1 0 1 0
```

If there is carry then we store 1 in carry flag.

② Parity Flag :- used to detect if transfered data is correct or not.

example-

Register —(due to voltage)→ RAM

Send data — lost/corrupt

parity = 0

0 1 1 0 1 0 1 1 ⟶ 0 1 1 0 1 0 0 1

(1) (2) (3) (4) (5) (1) (2) (3) (4)

if even parity :- If no's of 1 is even then parity bit is 0

parity bit ⇒ 1 (because no of one's is odd)

So when it is checked it shows that parity bit is not matching so RAM will ask for data again.

TA My NoteBook

Date / / Page no.

(3) Auxiliary Flag :-

(1) (1)
0100 1100
1100 1110

1110

4 bits is known as nibble, and whenever a carry comes just after a nibble then the bit in auxiliary flag becomes to '1'. and '0' if no carry after nibble.

(4) zero Flag :- if the answer is ~~zero~~ 0 after performing any operation then zero flag becomes true or we can say its bit ~~do~~ becomes '1'.

(5) Sign Flag :- As the name suggest this is used to memorize the sign of the answer. means if the answer is ~~zero~~ -ve or +ve.

For -ve, Sign flag becomes 1
+ve, Sign Flag becomes 0.

DELTA My NoteBook

Page no.

Date / /

⑥ Trap Flag :-

⑦ Interrupt Flag :- # Signal generated by h/w or s/w for CPU

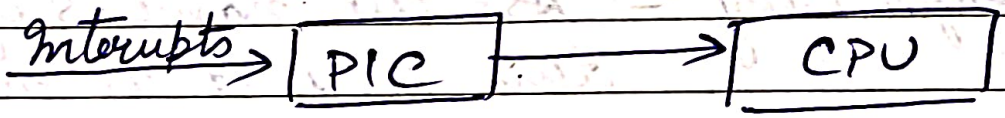
interrupt



when IF bit is true and a interrupt comes then CPU will handle interrupt first

Same if IF bit is false then CPU will ignore interrupt and finish its current work first.

→ programmable interrupt controller



I4 → I3 → I2 → I1

queue

- PIC constantly check if interupt flag is true ('1'). then it sends interupts to CPU.

- Queue is managed according to priority of interupts.

DELTA My NoteBook

Date / /

Page no.

(8) Direction Flag :- As the name suggest it is responsible for the direction in which data should be read.

false → "iLoveAssembly"

Tene → "ylbmessAeveLi"

(9) overflow flag :-

n1 * n2 = answer more than 16 bits
n1 + n2 = 17 bits

whenever there is overflow in data answer after performing any operation then overflow flag turns true.

⇒ IP/PC Register (instruction pointer/ program counter register).

it holds the address of the next instruction to be executed.

⇒ IR (instruction Register).

it holds the instruction that is in CPU

DELTA My NoteBook

Date / /

Page no.

example- Assembly language is very powerful as it gives direct access till Registers whereas C, C++, Java doesn't allow.

~~those~~

High level.

Instruction Groups:-

① Data movement instructions

mov rax, bx

lda 89

mov au, 19

mov au 89.

② ~~Airthmetic~~ Arithmetic / logic instruction.

② ① ③ → Sequence

add au, 33

add bx, [100]

and au, 23

Date / / Page no.

③ Program control Group.

cmp au, 12
↓
compare

jne 100
↑
jump not equals to

means:- jump to 100 address if zero flag ≠ 1

④ Special Instructions

cli → clear interrupt flag.

sti → Set interrupt flag.

Assembly language Prog.
download Dosbox, Nasm
which converts AL into ML.

Assembler.

Cmnds:- (i) mount c c:\assembly.
(ii) C:
(iii) nasm first.asm -o fir.com -l firstlist.lst
(iv) type firstlist.lst → program with form of list.
(v) afd fir.com.

DELTA My NoteBook

Versions:- Dosbox :— 0.74-3
Nasm :— 2.15.05
AFD → unofficial (github).

Date / /

(1) First program.

```
[org 0x100]

mov ax, 8
mov bx, 1
add ax, bx
mov bx, 4

mov ax, 0x4c00
int 0x21
```

(2) write a prog to in assembly lang that calculates the sum of square of 2 and 3 by adding through Registers.

```
mov ax, 2
add ax, 2
mov bx, 3
add bx, 3
add bx, 3
add ax, bx
```

Date / / Page no.

Segmented Memory model

old processors ⟶ new processors.

⇒ 8080, 8085 — 8088

Address bus ← 8 bit — 16 bit ⟹ (~~16~~ bit address bus)

$2^{16} = 65536$ memory locations

⇒ 80×86 — 80×64

32 bit (32 bit address bus, 32 bit data bus) — 64 bit (48 bit address bus)

* Why we need Segmented memory

compatibility — memory Increase

64KB → ~~1MB using 20 bit Address bus~~

* Demerit of 8080, 8085

8 bit

16 bit address bus

$2^{16} = 65536$ memory allocations

↓

64KB ← which is very less

* Note:—

Program

code | data | Stack

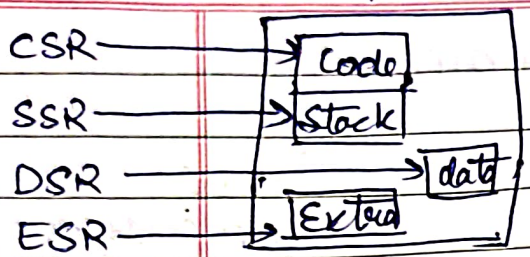
we divided memory into Small pieces of size 64KB, these pieces are called Segment

DELTA My NoteBook

Date

Segment Registers

Page no.



CSR :- Code Segment Register

SSR :- stack Segment R (Starting add of stack)

DSR :- Data Segment R
(stores add of Data Segment)

ESR :- Extra Segment R
(stores add of Extra Segment)

offset Registers

offset :=> Distance b/w starting add of segment and desired location.

CSR	00010	1	IP
DSR	00030	3	BP
SSR	00010	3	SP
ESR	00030	3	

SP ↑ Segment pointer

SI/DI ↑ Source index/ Destination index

Date / / Page no.

To Generate physical address from SR and OR.

Seg Reg add + zero ← at last zero & off Reg.

(+) Add both

physical address.

Ques1- example Question

if CS = 1234 H
IP = 0022H then find physical address.

CS ⇒ 12340H ⇒ ~~0001 0010 0011 0100 0000~~
IP ⇒ 00022H ↓

0001 0010 0011 0100 0000
0000 0000 0000 0010 0010

0001 0010 0011 0110 0010

phy add → 12362 H

Ques2- what would be the offset to map the physical address location 003C3H if the content of seg Register are 003AH.

Segment pointer.

phy = seg + off
phy − seg = off

DELTA My NoteBook

Date / /

Page no.

phy = 003C3H

Seg = 003A0H

```
                                    1
0000  0000  0011  1X600 0011
0000  0000  0011  1·0X0 0000
(-)
0000  0000  0000  0016 0011
```

00023H → off address

0023H

Ques 3: Seg = FF01H → FF010H

off = FF02H → 0FF02H

```
1111  1111  0000  0001  0000
0000  1111  1111  0000  0010
(1) 0000  1110  1111  0001  0010
```

↑ due to this extra bit, now it became 21 bit addrs.

we ignore this extra bit.

Date / / Page no.

Addressing Modes

The allowed approaches / methods in which we may access the RAM / Register.

1. Mediate mode → mov ax, 8
2. Direct Accen mode
3. Base Register Indirect mode
4. Base Register Indirect + offset mode.
5. Base Register + indeu mode.
6. Indeu Register indirect mode.
7. Indeu register indirect + offset mode
8. Base + indeu + offset mode.

Not allowed instruction in AL

(i) Size mismatch

mov 16 bit, 8 bit ✗ mov 16 bit, 16 bit
mov 8 bit, 16 bit ✗

mov ax, bl ✗ mov ax, bx

(ii) memory to memory

mov var1, var2 ✗ Slow (Two add will be gen.)

mov var1, ax ✓ Fast (only one add will be generated)
mov ax, bx ✓ Fastest

My NoteBook

Date / /

Page no.

(iii) offset Subtraction

example :- Array with starting add + index. (- index) is illegal

(2) Direct Access mode.

; Declaring variables in Assembly language.

define word.

var1 : dw 10 33; W ⇒ 16 bit

var2 : db 39; B ⇒ byte

define byte.

Assembler converts name of variable to memory address.

[ ] = value at the address.

assembler converts name of variable to memory address.

address of var1 = 99 value of var1 = 17

mov ax, var1 ⟶ mov ax, 99 (address)

mov ax, [var1] ⟶ mov ax, [99] → 17

Date / /

Page no.

w = word = 16 bit
b = byte = 8 bit

```
var1 : dw 10
var2 : db 20
```

(3) (4) (5) Base Register Indirect mode.

Ques:→ write a assembly Program that adds the numbers 5, 10, 15, 20

```
[org 0x100]
mov ax, 0
mov bx, num
add ax, [bx]
add ax, [bx+2]
add ax, [bx+4]
add ax, [bx+6]

mov ax, 0x4c00
int 0x21

num: dw 5, 10, 15, 20
```

```
mov ax, [bx]
mov ax, [bx+8]
mov ax, [bx+si]
```

(6) Index Register Indirect mode.

ex:→ mov ax, [si]

we put [ ] brackets around the index Register.

TA My NotesBook

Date / /

Page no.

(7) Index Register Indirect + offset.

eg:- mov ax, [si+2]

(8) Base + index + offset mode.

eg:- mov ax, [bx+si+2]

add ax, [bx] mov si, 2
add ax, [bx+si]
add ax, [bx+si+2]

ques:- write a assembly prog that adds the numbers 5, 10, 15

---

mov ax, 0
mov si, num
add ax, [si]
add ax, [si+2]
add ax, [si+4]

---

num : dw 5, 10, 15

Date / /. Page no.

Assembly Program

Registers/memory → of basics/memory/R access.

commands → if/else loop

#Branching in Assembly Lang

- transfer/shifting of control is called as branching

Statement 1
Statement 2
Statement 3
end.

- JUMP
  - conditional
  - unconditional